

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: ЗАДАЧА КОММИВОВАЖЕРА

Студент гр. 4345

Пикалев Н.Г.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритмы решения задачи коммивояжёра.

Постановка задачи

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

n — количество городов ($5 \leq n \leq 15$).

Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

Вариант 8. Точный метод: динамическое программирование (не МВиГ), итеративная реализация. Приближённый алгоритм: АЛШ-2.

Выполнение работы.

Решение задачи коммивояжёра методом полного перебора всех возможных маршрутов требует проверки факториального количества перестановок $O(N!)$, что делает такой подход неприменимым на практике даже для графов небольшой размерности. Для преодоления этой проблемы в данной работе были реализованы два алгоритма: точный метод на основе итеративного динамического программирования (алгоритм Хелда-Карпа) и приближённый эвристический метод АЛШ-2.

Для обеспечения независимости тестирования от ручного ввода данных был создан вспомогательный файл `matr_generator.py`. Программа запрашивает у пользователя количество городов (от 5 до 15) и тип матрицы (симметричная или асимметричная). Для симметричной матрицы веса генерируются только в верхнем треугольнике и зеркально копируются в нижний. Для асимметричной матрицы каждый элемент заполняется независимо. С вероятностью 10% ребро между двумя вершинами отсутствует (вес 0), что позволяет тестировать алгоритмы на несвязных графах. Сгенерированная матрица сохраняется в текстовый файл `matrix.txt`, который затем используется в программах.

Для выполнения точного решения задачи был создан файл `tsp_dp.py`. В нём реализован алгоритм Хелда-Карпа в итеративной форме. В отличие от рекурсивных вариантов, итеративный алгоритм заполняет таблицу состояний снизу-вверх, последовательно перебирая числовые значения битовых масок от 1 до $2^N - 1$.

Основными структурами данных являются двумерный массив, хранящий минимальную стоимость достижения каждой вершины при заданном подмножестве посещённых городов, и массив аналогичного размера, фиксирующий предшественников для последующего

восстановления оптимального пути. Базовым состоянием является начало пути в стартовом городе с нулевой стоимостью.

На каждой итерации для всех достижимых вершин текущей маски проверяются рёбра в ещё не посещённые города. При нахождении более короткого пути обновляется значение в таблице динамики и фиксируется предшественник. После завершения обхода всех масок оптимальный замкнутый маршрут восстанавливается в обратном направлении с использованием сохранённых связей в массиве. Если ни один из городов не позволяет вернуться в стартовую вершину, выводится сообщение "no path".

Временная сложность итеративного алгоритма Хелда-Карпа оценивается как $O(2^N \cdot N^2)$, поскольку количество уникальных подмножеств вершин составляет 2^N , а для каждого состояния выполняется перебор текущих и целевых вершин, дающий квадратичный множитель. Сложность по памяти составляет $O(2^N \cdot N)$, что обусловлено выделением памяти под матрицы.

В качестве приближённого метода был создан файл alsh2.py, реализующий алгоритм лучшего шага второго порядка (АЛШ-2). Алгоритм осуществляет последовательное построение маршрута из стартовой вершины, на каждом шаге выбирая следующую вершину по критерию минимума функции $f = s + L$, где s — вес ребра до кандидата, а L — нижняя оценка стоимости оставшегося пути.

Оценка L в АЛШ-2 вычисляется функцией `calc_L_alsh2` на основе полсуммы весов легчайших рёбер и включает три компонента: минимальное исходящее ребро из текущего конца пути в не посещённые вершины, минимальное входящее ребро в стартовую вершину из не посещённых вершин, а также для каждой не посещённой вершины — сумму её минимального входящего и минимального исходящего рёбер.

Временная сложность алгоритма АЛШ-2 оценивается как $O(N^3)$. Внешний цикл выполняется N раз, на каждом шаге перебираются не

посещённые кандидаты ($O(N)$), для каждого из которых вызывается процедура расчёта нижней оценки L , требующая $O(N)$ операций для анализа рёбер оставшихся вершин. Сложность по памяти составляет $O(N)$, так как память требуется только для хранения массива посещённых вершин и текущего пути.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование программы tsp_dp.py

Ввод	Результат	Комментарий
5 0 1 13 23 7 12 0 15 18 28 21 29 0 33 28 23 19 34 0 38 5 40 7 39 0	78 0 4 2 3 1 0	Пример из условия задачи
4 0 18 19 5 42 0 43 25 4 38 0 38 27 9 18 0	61 0 3 1 2 0	Случайно сгенерированная матрица
5 0 34 29 15 0 34 0 30 35 45 29 30 0 8 6 15 35 8 0 0 0 45 6 0 0	108 0 3 2 4 1 0	Случайно сгенерированная симметричная матрица

Таблица 2 – Тестирование alsh2.py

Ввод	Результат	Комментарий
3 0 1 0 1 0 1 0 1 0	no path	Пример из условия задачи
5 0 10 7 50 3 34 0 15 49 41 30 1 0 5 17 42 45 33 0 43 12 4 13 38 0	69 0 4 1 2 3 0	Случайно сгенерированная матрица
4 0 46 48 43 46 0 29 29 48 29 0 9 43 29 9 0	127 0 3 2 1 0	Случайно сгенерированная симметричная матрица

Выводы

В рамках данной лабораторной работы были успешно разработаны и проанализированы точный и эвристический алгоритмы для нахождения кратчайшего замкнутого маршрута в задаче коммивояжера. Точный подход реализован посредством итеративного алгоритма Хелда-Карпа на основе динамического программирования, в то время как в качестве приближенного метода был выбран алгоритм лучшего шага (АЛШ-2). Дополнительно была создана вспомогательная утилита для генерации симметричных и асимметричных весовых матриц со случайными ребрами, обеспечивающая независимое сохранение тестовых данных в текстовый файл и их последующее считывание для решения.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл: matr_generator.py

```
import random

def generate_matrix(n, symmetric=False):
    matrix = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if i != j:
                if symmetric:
                    if j > i:
                        if random.random() > 0.1:
                            val = random.randint(1, 50)
                            matrix[i][j] = val
                            matrix[j][i] = val
                else:
                    if random.random() > 0.1:
                        matrix[i][j] = random.randint(1, 50)

    with open("matrix.txt", "w") as f:
        f.write(str(n) + "\n")
        for row in matrix:
            f.write(" ".join(map(str, row)) + "\n")

    print(f"Матрица {n}x{n} ({'симметричная' if symmetric else
'асимметричная'}) сохранена в matrix.txt")

if __name__ == "__main__":
    n = int(input("Введите количество городов (5-15): "))
    sym = input("Матрица симметричная? (y/n): ").strip().lower()
    == 'y'
    generate_matrix(n, symmetric=sym)
```

Файл: tsp_dp.py

```
def solve_tsp_dp(n, graph):
    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]

    dp[1][0] = 0

    print("Итеративное заполнение таблицы ДП:")
    for mask in range(1, 1 << n):
        bin_mask = bin(mask)[2:].zfill(n)
        print(f"\nОбработка маски {bin_mask} (десятичное
значение: {mask}):")
        for u in range(n):
            if dp[mask][u] == INF:
                continue
```

```

        print(f"    Город {u} достигим, стоимость пути:
{dp[mask][u]}")
        for v in range(n):
            if not (mask & (1 << v)) and graph[u][v] > 0:
                new_mask = mask | (1 << v)
                new_mask_bin = bin(new_mask)[2:].zfill(n)
                new_cost = dp[mask][u] + graph[u][v]
                print(f"        Проверяем переход {u} -> {v}
(ребро: {graph[u][v]})")
                if new_cost < dp[new_mask][v]:
                    print(f"                        Обновляем
dp[маска={new_mask_bin}][город={v}]: {dp[new_mask][v]} -> {new_cost}
через город {u}")
                    dp[new_mask][v] = new_cost
                    parent[new_mask][v] = u
                else:
                    print(f"        Переход невыгоден (текущая
стоимость {dp[new_mask][v]} <= {new_cost})")

        full_mask = (1 << n) - 1
        min_cost = INF
        last_city = -1

        print("\nПроверка возврата в стартовый город 0:")
        for u in range(1, n):
            if graph[u][0] > 0 and dp[full_mask][u] != INF:
                cost = dp[full_mask][u] + graph[u][0]
                print(f"    Путь через город {u} -> 0: стоимость
{dp[full_mask][u]} + {graph[u][0]} = {cost}")
                if cost < min_cost:
                    min_cost = cost
                    last_city = u

        if min_cost == INF:
            return None, None

        print("\nВосстановление оптимального пути:")
        path = []
        mask = full_mask
        city = last_city

        while city != 0:
            path.append(city)
            print(f"        Добавляем    город    {city}    (маска
{bin(mask)[2:].zfill(n)})")
            prev_city = parent[mask][city]
            mask = mask ^ (1 << city)
            city = prev_city

        path.reverse()
        path = [0] + path + [0]

        return min_cost, path

def main():
    graph = []

```

```

with open("matrix.txt", "r") as f:
    lines = f.readlines()
    n = int(lines[0].strip())
    for i in range(1, n + 1):
        graph.append(list(map(int,
lines[i].strip().split()))))

result, path = solve_tsp_dp(n, graph)

print("\nРезультат:")
if result is None:
    print("no path")
else:
    print(result)
    print(' '.join(map(str, path)))

if __name__ == "__main__":
    main()

```

Файл alsh2.py

```

def calc_L_alsh2(n, graph, visited, path):
    start = path[0]
    end = path[-1]

    min_out_end = float('inf')
    for v in range(n):
        if not visited[v] and graph[end][v] > 0:
            min_out_end = min(min_out_end, graph[end][v])

    min_in_start = float('inf')
    for v in range(n):
        if not visited[v] and graph[v][start] > 0:
            min_in_start = min(min_in_start, graph[v][start])

    unvisited_sum = 0
    for u in range(n):
        if not visited[u]:
            min_out_u = float('inf')
            for v in range(n):
                if (not visited[v] or v == start) and u != v and
graph[u][v] > 0:
                    min_out_u = min(min_out_u, graph[u][v])

            min_in_u = float('inf')
            for v in range(n):
                if (not visited[v] or v == end) and u != v and
graph[v][u] > 0:
                    min_in_u = min(min_in_u, graph[v][u])

            if min_out_u == float('inf'): min_out_u = 0
            if min_in_u == float('inf'): min_in_u = 0
            unvisited_sum += min_out_u + min_in_u

    if min_out_end == float('inf'): min_out_end = 0
    if min_in_start == float('inf'): min_in_start = 0

```

```

        return (min_out_end + min_in_start + unvisited_sum) / 2

def alsh2(n, graph):
    visited = [False] * n
    path = [0]
    visited[0] = True
    total_cost = 0
    curr = 0

    print("алгоритм - Аллш-2")
    for i in range(n - 1):
        next_city = -1
        best_f = float('inf')
        best_s = 0
        best_L = 0

        print(f"\nШаг {i + 1}")
        print(f"Текущий город в пути: {curr}")

        for v in range(n):
            if not visited[v] and graph[curr][v] > 0:
                s = graph[curr][v]

                visited[v] = True
                path.append(v)
                L = calc_L_alsh2(n, graph, visited, path)
                path.pop()
                visited[v] = False

                f = s + L
                print(f"    Проверка {curr} -> {v}: s + L = {s} +
{L} = {f}")

                if f < best_f:
                    best_f, best_s, best_L = f, s, L
                    next_city = v

        print(f"    Выбран город: {next_city}")
        print(f"    Лучшее значение f: s + L = {best_s} + {best_L}
= {best_f}")

        if next_city == -1:
            return None, None

        path.append(next_city)
        visited[next_city] = True
        total_cost += graph[curr][next_city]
        print(f"    Текущая стоимость накопленного пути:
{total_cost}")
        curr = next_city

    if graph[curr][0] > 0:
        total_cost += graph[curr][0]
        path.append(0)
        print(f"\nВозврат в стартовый город 0 за
{graph[curr][0]}")
        return total_cost, path
    return None, None

```

```

def main():
    graph = []
    with open("matrix.txt", "r") as f:
        lines = f.readlines()
        n = int(lines[0].strip())
        for i in range(1, n + 1):
            graph.append(list(map(int,
lines[i].strip().split()))))

    result, path = alsh2(n, graph)

    print("\nРезультат:")
    if result is None:
        print("no path")
    else:
        print(result)
        print(' '.join(map(str, path)))

if __name__ == "__main__":
    main()

```